

GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

LABORATORY REPORT



AGILE SOFTWARE DEVELOPMENT LAB

Lab No. 5

BESE III YEAR, VI-SEM

Submitted By:

Prabin Thapa

Submitted To:

Er. Rajendra Bahadur Thapa

Date: July 4, 2026

NAME OF EXPERIMENT: Implementation of CI/CD Pipelines Using GitHub Actions and Jenkins

AIM: To understand and implement **Continuous Integration (CI)** and **Continuous Deployment/Delivery (CD)** pipelines using **GitHub Actions** and **Jenkins** for automating the software build, testing, and deployment process..

Objectives:

1. To understand the concepts of Continuous Integration (CI) and Continuous Deployment/Delivery (CD).
2. To study the architecture and workflow of CI/CD pipelines.
3. To implement automated workflows using GitHub Actions.
4. To configure and execute build jobs using Jenkins.
5. To automate testing and deployment processes.
6. To understand the benefits of automation in software development.
7. To improve software quality through continuous integration and deployment

THEORY:

Introduction to CI/CD

Continuous Integration (CI) is a software development practice where developers frequently integrate code changes into a shared repository. Each integration automatically triggers a build and testing process to detect errors early.

Continuous Delivery (CD) ensures that the software is always in a deployable state by automating the release process.

Continuous Deployment extends Continuous Delivery by automatically deploying successfully tested code to production without manual intervention.

CI/CD pipelines reduce manual effort, improve software quality, and accelerate software delivery.

1. GitHub Actions

GitHub Actions is GitHub's built-in automation platform that enables developers to automate workflows directly from a GitHub repository. Workflows are defined using YAML files and are triggered by events such as code pushes, pull requests, or scheduled tasks.

Features:

- Automatic build and testing
- Workflow automation
- Event-based execution

- Integration with GitHub repositories
- Multi-platform support

2. Jenkins

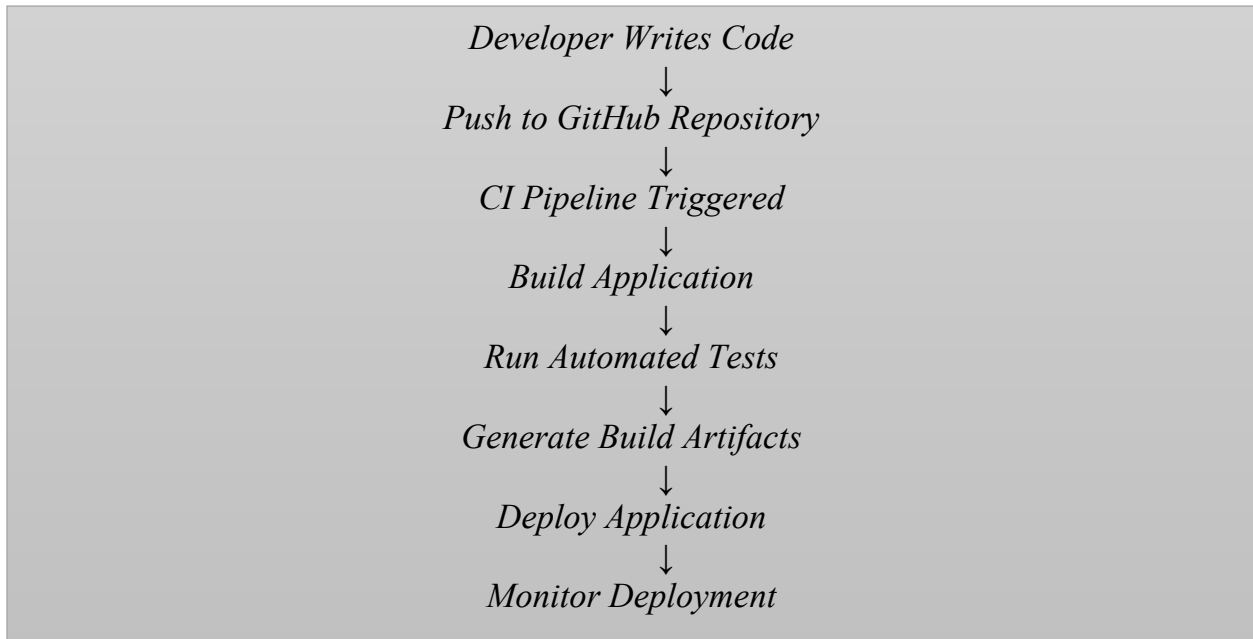
Jenkins is an open-source automation server widely used for implementing CI/CD pipelines. It automates building, testing, and deploying software projects through jobs or pipelines.

Features:

- Automated builds
- Continuous integration
- Pipeline support
- Plugin ecosystem
- Integration with Git, Docker, Maven, Gradle, and other tools

CI/CD Pipeline Workflow

A typical CI/CD pipeline follows these steps:



Advantages of CI/CD

- Automates repetitive development tasks.
- Detects bugs early through automated testing.

- Reduces manual deployment errors.
- Improves software quality and reliability.
- Enables faster software delivery.
- Encourages collaboration among developers.

Algorithm

1. Create a GitHub repository.
2. Develop the application and push the source code to GitHub.
3. Create a GitHub Actions workflow file.
4. Configure build and testing steps in the workflow.
5. Commit and push the workflow file.
6. Verify that GitHub Actions automatically executes the workflow.
7. Install and configure Jenkins.
8. Create a Jenkins job or pipeline.
9. Connect Jenkins to the GitHub repository.
10. Configure build and test steps.
11. Execute the Jenkins pipeline.
12. Verify successful build and deployment.
13. End.

CODE: Implementation

1. GitHub Actions Workflow

Create the file:

```
.github/workflows/python-ci.yml
```

Workflow File

```
name: Python CI

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install Dependencies
        run: |
          python -m pip install --upgrade pip

      - name: Run Python Program
        run: |
          python app.py
```

Sample Python Program (app.py)

```
def greet():
    print("Hello, CI/CD Pipeline!")

if __name__ == "__main__":
    greet()
```

GitHub Actions Output

Workflow Started

✓ Checkout Repository

✓ Setup Python

✓ Install Dependencies

✓ Run Python Program

Hello, CI/CD Pipeline!

Workflow Completed Successfully

2. Jenkins Pipeline

Jenkinsfile:

```
pipeline {
  agent any

  stages {

    stage('Checkout') {
      steps {
        git 'https://github.com/username/repository.git'
      }
    }

    stage('Build') {
      steps {
        echo 'Building Project...'
      }
    }

    stage('Test') {
      steps {
        sh 'python app.py'
      }
    }

    stage('Deploy') {
      steps {
        echo 'Deploying Application...'
      }
    }
  }
}
```

Jenkins Console Output

```
Started by user  
Checking out source code...  
Building Project...  
Running Tests...  
Hello, CI/CD Pipeline!  
Deploying Application...  
Finished: SUCCESS
```

Git Commands Used

```
git init  
git add .  
git commit -m "Added CI/CD pipeline"  
git push origin main
```

Pipeline Stages

Stage	Description
Checkout	Downloads the source code from GitHub
Build	Compiles or prepares the project
Test	Executes automated tests
Deploy	Deploys the application after successful testing

Sample Output

GitHub Actions

Status: ✓ Success

Build Completed

Tests Passed

Deployment Completed

Jenkins

Build #1

Status: SUCCESS

Pipeline Executed Successfully

RESULT

The implementation of the CI/CD pipeline using GitHub Actions and Jenkins was successfully carried out. Automated workflows were created to build, test, and execute the application whenever changes were pushed to the GitHub repository. The pipelines executed successfully, demonstrating the effectiveness of continuous integration and deployment practices.

CONCLUSION

This experiment provided practical knowledge of implementing Continuous Integration (CI) and Continuous Deployment/Delivery (CD) using GitHub Actions and Jenkins. GitHub Actions automated the workflow directly from the GitHub repository, while Jenkins demonstrated pipeline-based automation for building, testing, and deploying applications. The implementation showed that CI/CD pipelines improve software quality by automating repetitive tasks, detecting errors early, reducing deployment time, and enabling faster and more reliable software delivery. These practices are essential components of modern DevOps and Agile software development.